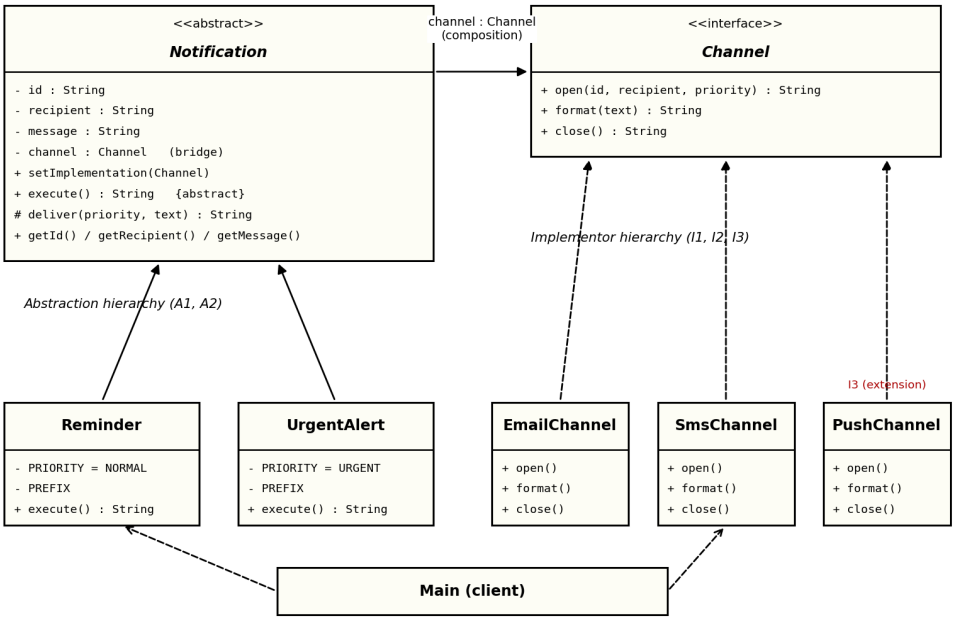
Assignment 3: Build a Small Java Application Using the Bridge Pattern

Name: Bekarys Zharylgassyn **Group:** SE-2526 **Topic:** B – Notifications **Repository:** https://github.com/veores/SDP-assignment-3 **Base commit:** f88c6548bc37cd78f46226d7384fc5e721f1a2da

1. Two independent dimensions and why Bridge fits

The first dimension is *what is sent*: Reminder (normal priority, text prefixed with "Reminder: ") and UrgentAlert (priority URGENT, text prefixed with "URGENT: "). The second is *how it is delivered*: EmailChannel (envelope with to/id/priority and a quoted body), SmsChannel (single-line text) and, after the extension, PushChannel (push envelope). Without Bridge every pair would need its own subclass (2 x 3 = 6 classes, growing with each new type or channel). With Bridge the abstraction Notification holds a reference to the Channel interface; each side grows on its own, giving 2 + 3 classes, and the channel can be replaced at runtime with setImplementation(...).

2. UML class diagram



3. Annotated code excerpts

Separation of responsibilities. Notification owns the workflow and domain data; Channel owns the rendering. Benefit: a new channel never touches notification logic.

```
protected String deliver(String priority, String text) {
    return channel.open(id, recipient, priority)
         + channel.format(text) + channel.close();
}
```

Meaningful names. Constants and methods say what they mean (PRIORITY, PREFIX, setImplementation, deliver). Benefit: the class reads without extra explanation.

```
private static final String PRIORITY = "URGENT";
private static final String PREFIX = "URGENT: ";
```

Small focused methods. Each execute() and each channel operation does one thing. Benefit: easy to test and to explain in the defense.

```
public String execute() {
    return deliver(PRIORITY, PREFIX + getMessage());
}
```

No duplication. The open-format-close workflow exists once in deliver(), shared by A1 and A2; Main.report() holds the shared PASS/FAIL printing. Benefit: a change to the workflow is made in one place.

```
private static void report(String id, boolean ok, String participants,
                           String details, String expected) {
    total++;
    if (ok) { passed++; }
    ...
```

Encapsulation of state. Fields are private; id, recipient and message are final and exposed through getters, and the channel can only change via setImplementation, which rejects null. Benefit: runtime switching cannot corrupt domain data.

```
private final String id;
private Channel channel;
public void setImplementation(Channel channel) {
    this.channel = Objects.requireNonNull(channel);
}
```

4. Extension, trade-off and comparison with Adapter

Extension. After committing the working I1/I2 version (base commit above), I added only PushChannel.java, which implements Channel, and updated Main with T6 and T7 plus the push samples (extension.diff: 2 files, 27 insertions). Notification, Reminder, UrgentAlert, Channel, EmailChannel and SmsChannel stayed unchanged; the existing abstractions work with PushChannel through the same interface.

Trade-off. Bridge adds an extra interface and an indirection layer, so a tiny program with one fixed channel becomes more complex than a single class; the Channel contract (open/format/close) must also be general enough for all channels, which limits channel-specific features.

Bridge vs Adapter. Bridge is designed up front to let abstraction and implementation vary independently. Adapter is applied afterwards to make an existing, incompatible class fit an expected interface. In this application both hierarchies are my own and were designed to be combined, so Bridge is the right choice; if I had to use a third-party notification class with a different method signature, I would wrap it in an Adapter that implements Channel.

Sources. Lecture 4, Bridge Pattern (course materials, 2026-2027); Gamma, Helm, Johnson, Vlissides, *Design Patterns*, Addison-Wesley, 1994; Martin, *Clean Code*, Prentice Hall, 2008; Refactoring Guru, Bridge pattern article.